VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

M.Sai Praneetha (1BF24CS163)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by M.Sai Praneetha **(1BF24CS163)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Pallavi C V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

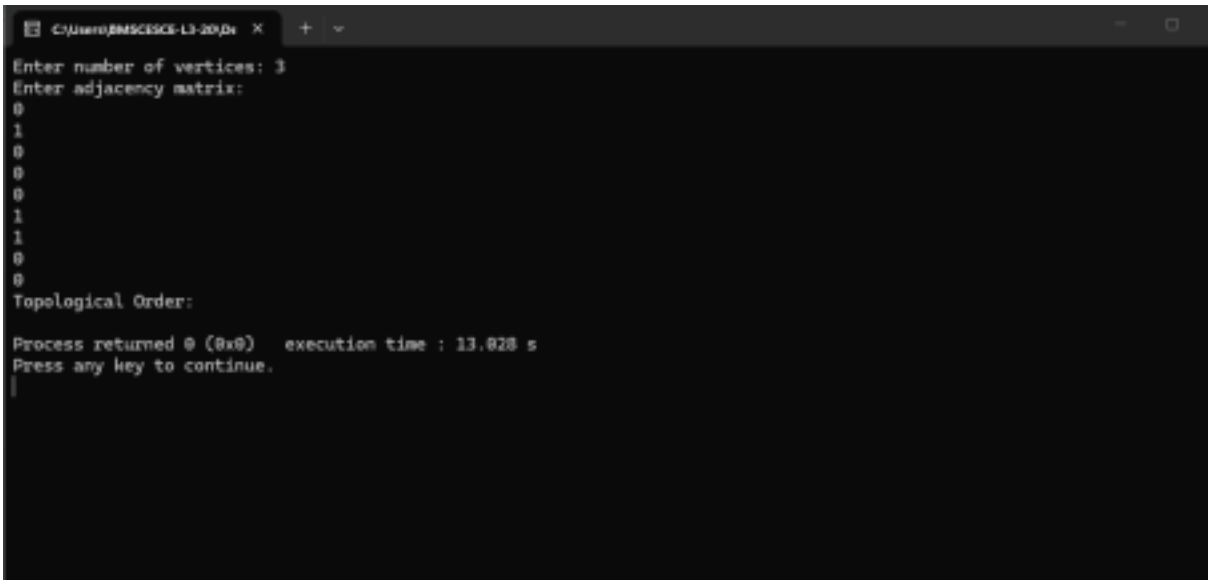
Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#define MAX 100
void topologicalSort(int graph[MAX][MAX], int n)
{
 int indegree[MAX] = {0};
 for(int i = 0; i < n; i++)
 {
 for(int j = 0; j < n; j++)
 {
 if(graph[i][j] == 1)
 indegree[j]++;
 }
 }
 int queue[MAX];
 int front = 0, rear = 0;
 for(int i = 0; i < n; i++)
 {
 if(indegree[i] == 0)
 queue[rear++] = i;
 }
 printf("Topological Order: ");
 while(front < rear)
 {
 int u = queue[front++];
 printf("%d ", u);
 for(int v = 0; v < n; v++)
 {
 if(graph[u][v] == 1)
 {
 indegree[v]--;
 if(indegree[v] == 0)
```

```
 queue[rear++] = v;
 }
 }
 }
 printf("\n");
}
int main()
{
 int n;
 printf("Enter number of vertices: ");
 scanf("%d", &n);
 int graph[MAX][MAX];
 printf("Enter adjacency matrix:\n");
 for(int i = 0; i < n; i++)
 {
 for(int j = 0; j < n; j++)
 {
 scanf("%d", &graph[i][j]);
 }
 }
```

OUTPUT:


C:\Users\BMSCEISCE-L3-20\Dc
Enter number of vertices: 3
Enter adjacency matrix:
0
1
0
0
0
1
1
0
0
Topological Order:
Process returned 0 (0x0) execution time : 13.028 s
Press any key to continue.

LeetCode:207 Course Schedule

207. Course Schedule

Solved

Medium | Topics | Companies | Hint

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [a_i, b_i]` indicates that you **must** take course `b_i` first if you want to take course `a_i`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

> **Input:** numCourses = 2, prerequisites = [[1,0]]
> **Output:** true
> **Explanation:** There are a total of 2 courses to take.
> To take course 1 you should have finished course 0. So it is possible.

Example 2:

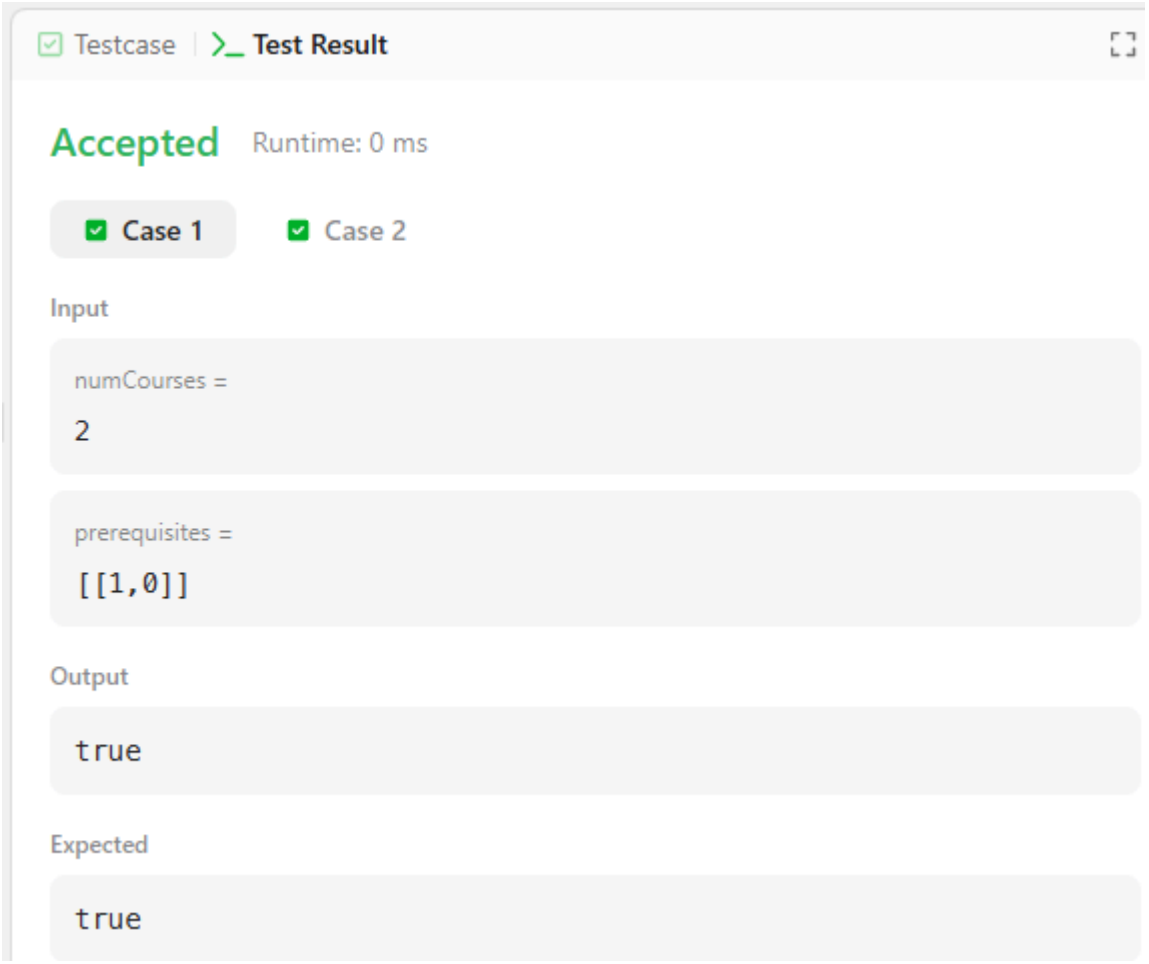
> **Input:** numCourses = 2, prerequisites = [[1,0],[0,1]]
> **Output:** false

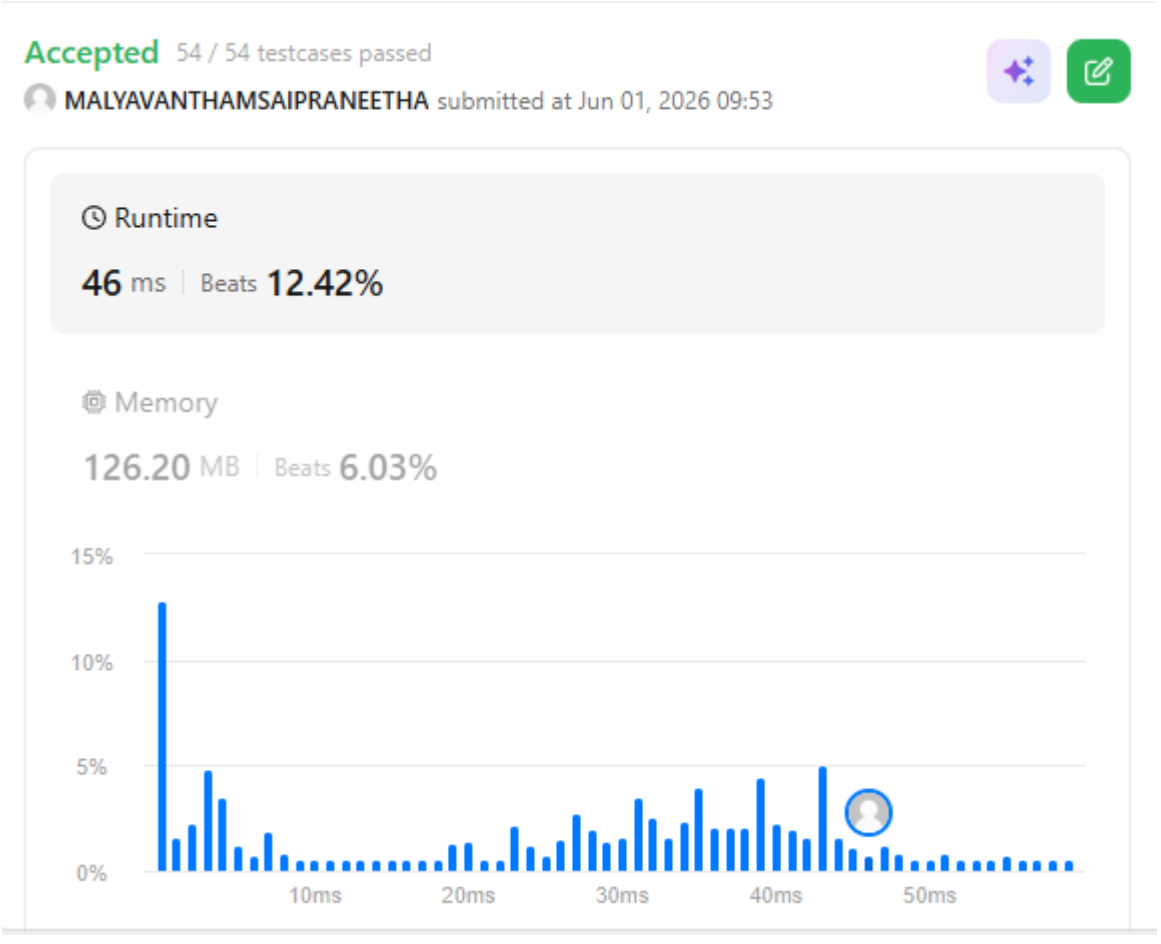
18.1K | 350

```c
#include <stdbool.h>
#include <stdlib.h>

bool canFinish(int numCourses, int** prerequisites,
               int prerequisitesSize,
               int* prerequisitesColSize) {

    int* indegree = (int*)calloc(numCourses, sizeof(int));

    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* size = (int*)calloc(numCourses, sizeof(int));

    for(int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(prerequisitesSize * sizeof(int));
    }
    for(int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        int prereq = prerequisites[i][1];

        graph[prereq][size[prereq]++] = course;
```

```
        indegree[course]++;
    }

    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for(int i = 0; i < numCourses; i++) {
        if(indegree[i] == 0)
            queue[rear++] = i;
    }

    int count = 0;

    while(front < rear) {
        int u = queue[front++];
        count++;

        for(int i = 0; i < size[u]; i++) {
            int v = graph[u][i];

            indegree[v]--;

            if(indegree[v] == 0)
                queue[rear++] = v;
        }
    }

    for(int i = 0; i < numCourses; i++)
        free(graph[i]);

    free(graph);
    free(indegree);
    free(size);
    free(queue);

    return count == numCourses;
}
```

OUTPUT:


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2
prerequisites =
[[1,0]]
Output
true
Expected
true


Accepted
54 / 54 testcases passed
MALYAVANTHAMSAIPRANEETHA submitted at Jun 01, 2026 09:53
Runtime
46 ms
Beats 12.42%
Memory
126.20 MB
Beats 6.03%
15%
10%
5%
0%
10ms
20ms
30ms
40ms
50ms

Lab program 2:

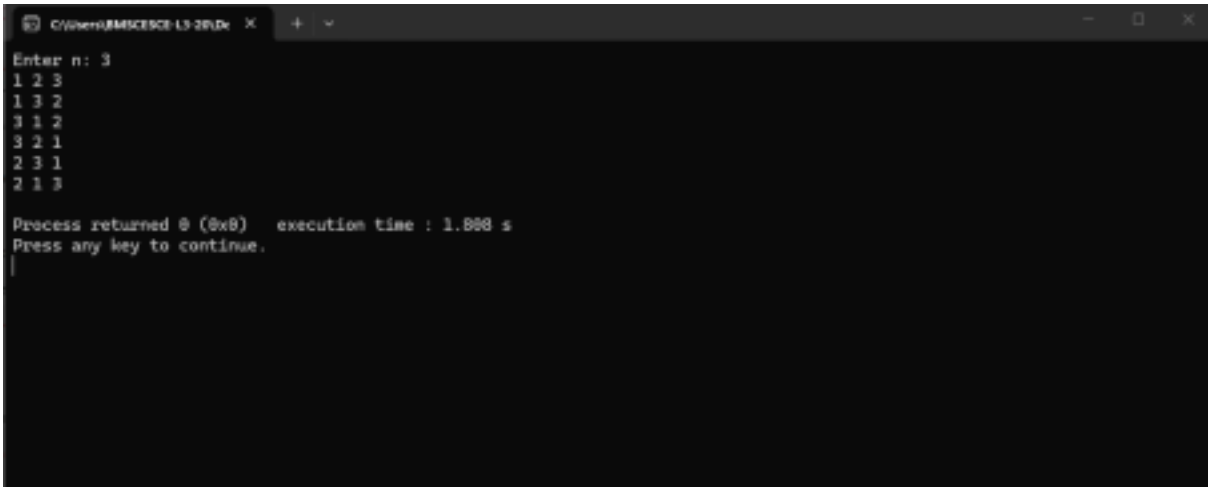
Implement Johnson Trotter algorithm to generate permutations.

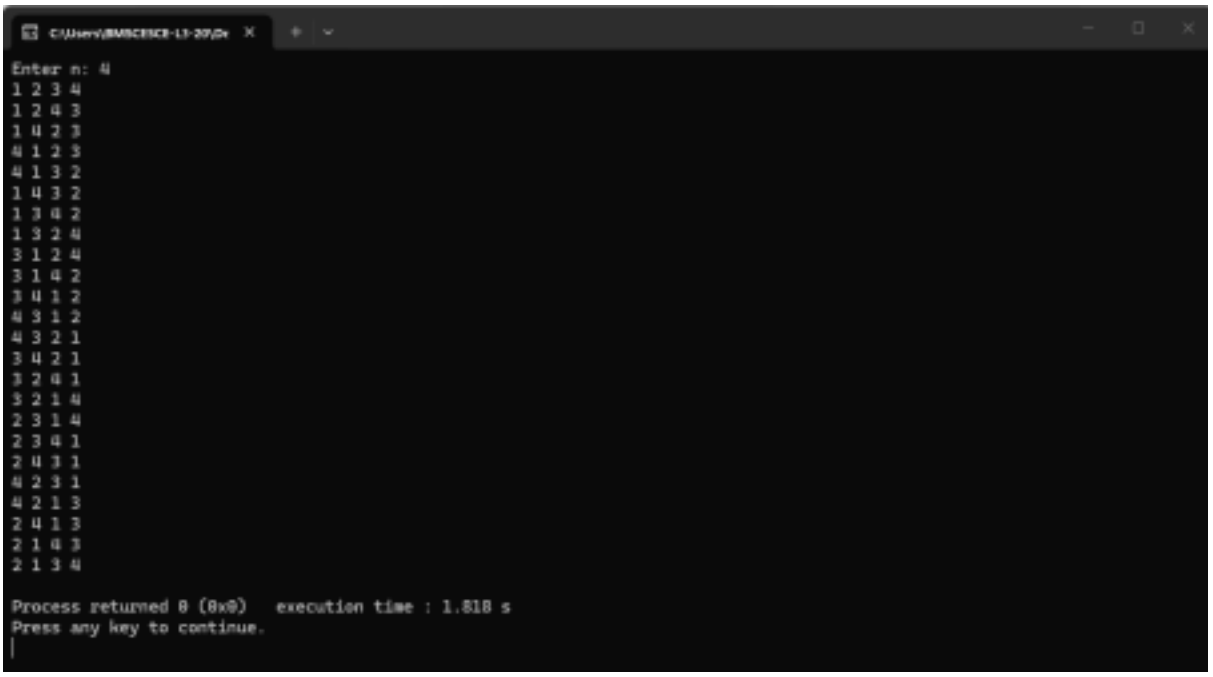
```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1
int getMobile(int a[], int dir[], int n)
 {
 int mobile = 0, mobile_prev = 0;
 for(int i = 0; i < n; i++)
 {
 if(dir[a[i]-1] == LEFT && i != 0)
 {
 if(a[i] > a[i-1] && a[i] > mobile_prev)
 {
 mobile = a[i];
 mobile_prev = mobile;
 }
 }
 if(dir[a[i]-1] == RIGHT && i != n-1)
 {
 if(a[i] > a[i+1] && a[i] > mobile_prev)
 {
 mobile = a[i];
 mobile_prev = mobile;
 }
 }
 }
 return mobile;
}
int searchPos(int a[], int n, int mobile)
 {
 for(int i = 0; i < n; i++)
 if(a[i] == mobile)
 return i;
```

```
 return -1;
}
void printPermutation(int a[], int n)
 {
 for(int i = 0; i < n; i++)
 printf("%d ", a[i]);
 printf("\n");
}
void johnsonTrotter(int n)
 {
 int a[n], dir[n];
 for(int i = 0; i < n; i++)
 {
 a[i] = i + 1;
 dir[i] = LEFT;
 }
 printPermutation(a, n);
 while(1)
 {
 int mobile = getMobile(a, dir, n);
if(mobile == 0)
 break;
 int pos = searchPos(a, n, mobile);
if(dir[mobile-1] == LEFT)  {
 int temp = a[pos];
 a[pos] = a[pos-1];
 a[pos-1] = temp;
 pos--;
 }
 else
 {
 int temp = a[pos];
 a[pos] = a[pos+1];
 a[pos+1] = temp;
 pos++;
```

```
 }
 for(int i = 0; i < n; i++)
 {
 if(a[i] > mobile)
 dir[a[i]-1] *= -1;
 }
 printPermutation(a, n);
 }
}
int main()
 {
 int n;
 printf("Enter n: ");
 scanf("%d", &n);
 johnsonTrotter(n);
 return 0;
}
```

OUTPUT:




Enter n: 4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4
Process returned 0 (0x0) execution time : 1.818 s
Press any key to continue.

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

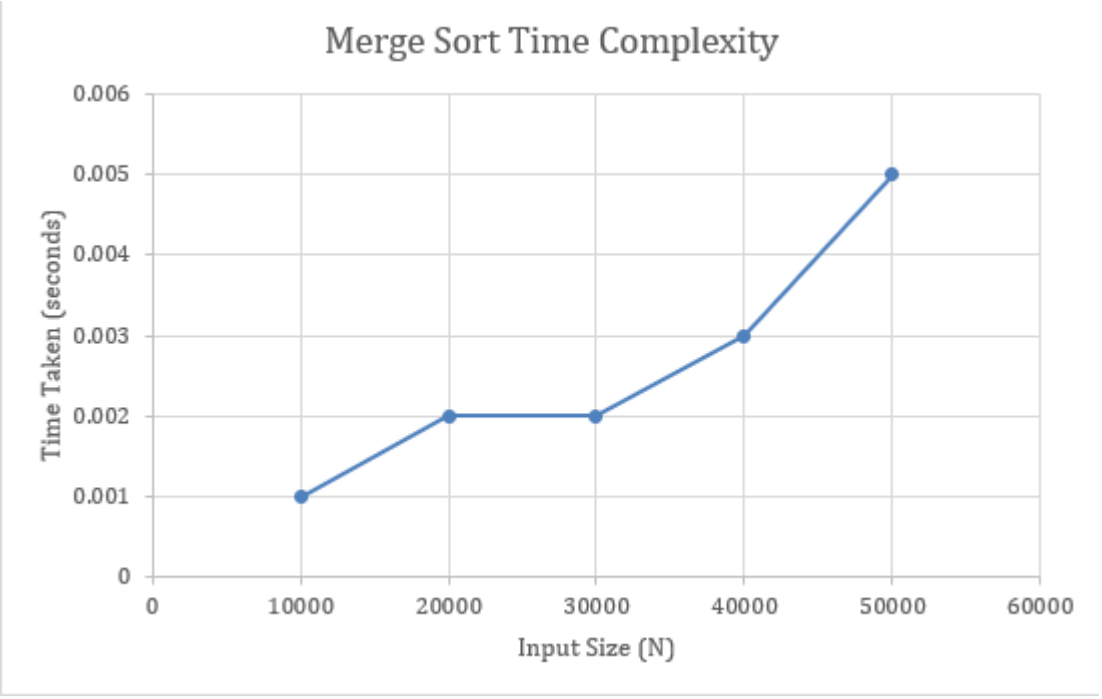
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int arr[], int low, int mid, int high)
{
   int i = low, j = mid + 1, k = 0;
   int temp[100000];

   while (i <= mid && j <= high)
   {
      if (arr[i] <= arr[j])
         temp[k++] = arr[i++];
      else
         temp[k++] = arr[j++];
   }

   while (i <= mid)
      temp[k++] = arr[i++];

   while (j <= high)
      temp[k++] = arr[j++];

   for (i = low, k = 0; i <= high; i++, k++)
      arr[i] = temp[k];
}

void mergeSort(int arr[], int low, int high)
{
```

```
    if (low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);

        merge(arr, low, mid, high);
    }
}
int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    srand(time(NULL));
    for (int i = 0; i < n; i++)
        arr[i] = rand() % 10000;
    clock_t start = clock();
    mergeSort(arr, 0, n - 1);
    clock_t end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Sorted elements:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\nTime taken = %f seconds\n", time_taken);
    return 0;
}
```

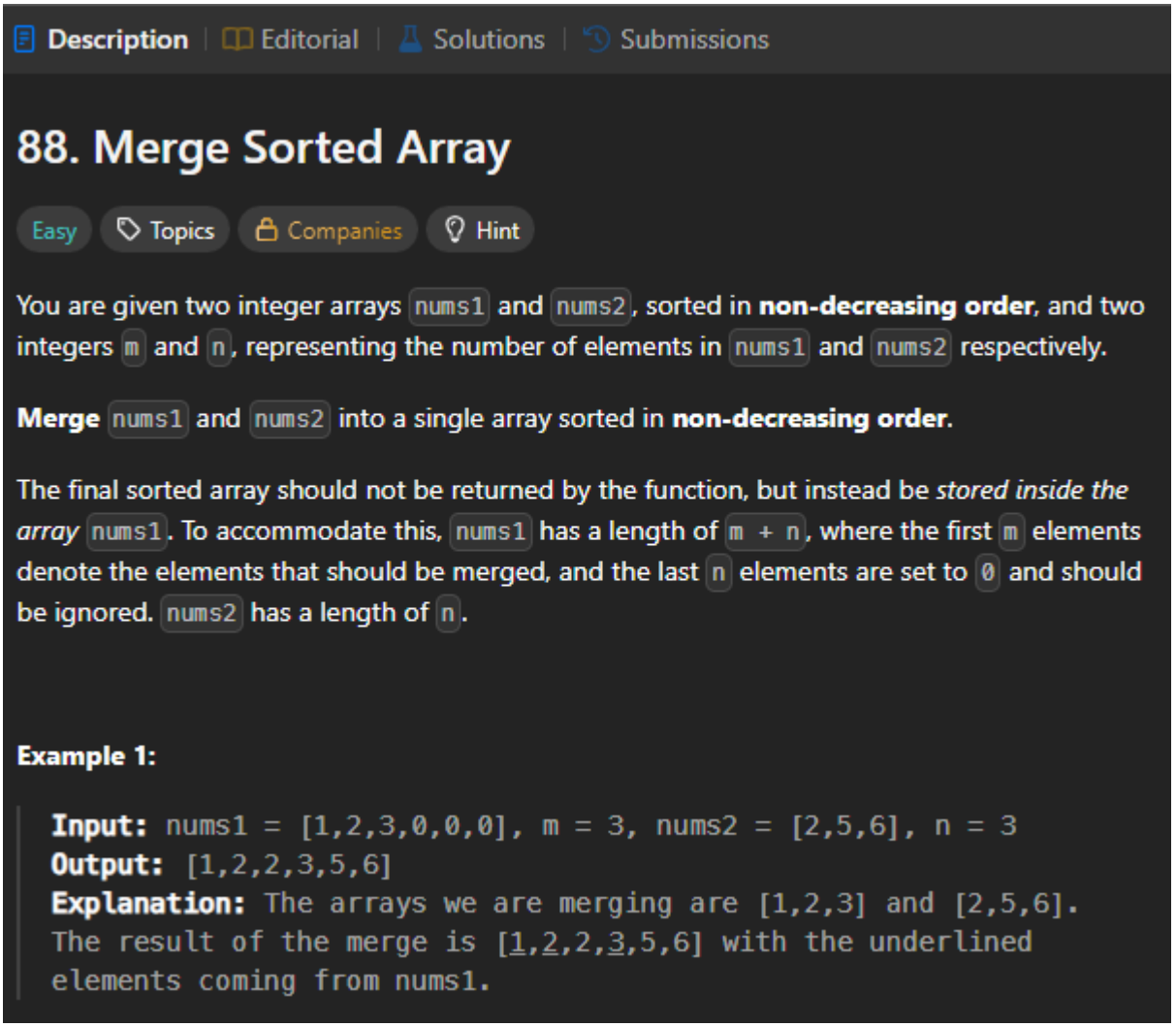
OUTPUT:

```
Output
Enter number of elements: 5
Sorted elements:
1076 1355 1363 1999 3923
Time taken = 0.000008 seconds
```

```
Output
Enter number of elements: 10
Sorted elements:
836 3415 4033 4044 4328 5669 7223 8182 9365 9421
Time taken = 0.000034 seconds
```



LeetCode:88 Merge Sorted Array



Description | Editorial | Solutions | Submissions

88. Merge Sorted Array

Easy | Topics | Companies | Hint

You are given two integer arrays `nums1` and `nums2`, sorted in **non-decreasing order**, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in **non-decreasing order**.

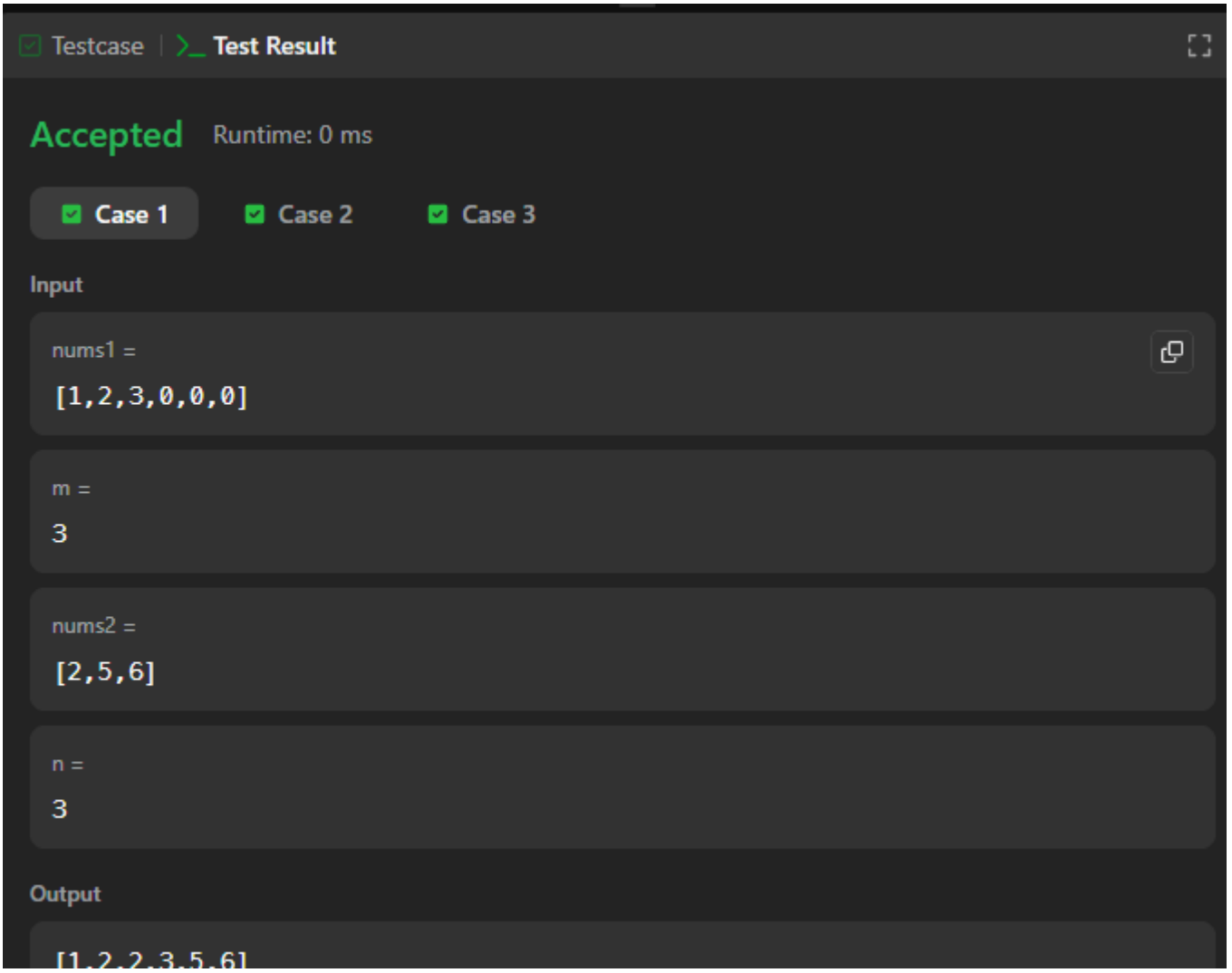
The final sorted array should not be returned by the function, but instead be *stored inside the array* `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to `0` and should be ignored. `nums2` has a length of `n`.

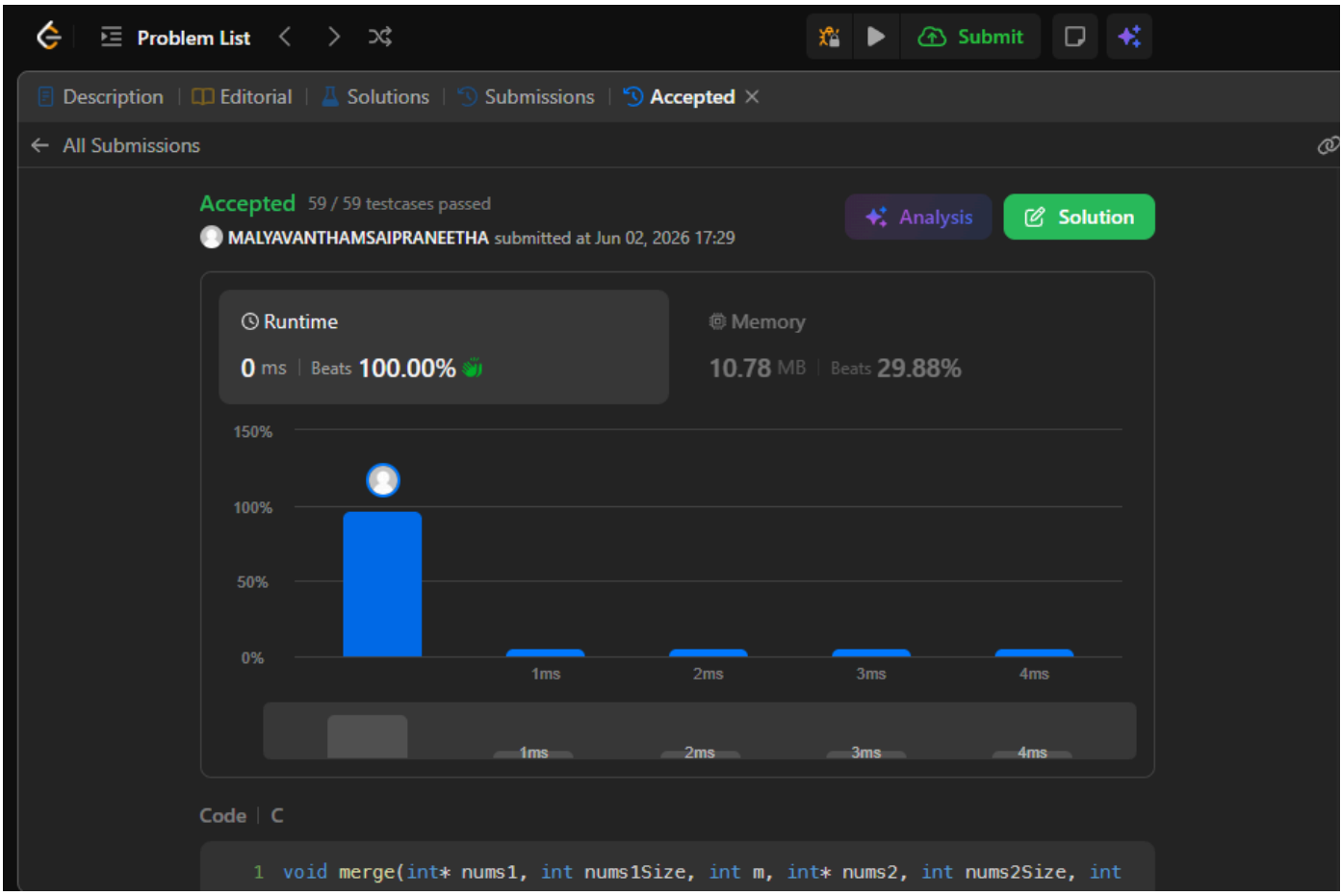
Example 1:

> **Input:** nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3
> **Output:** [1,2,2,3,5,6]
> **Explanation:** The arrays we are merging are [1,2,3] and [2,5,6].
> The result of the merge is [1,2,2,3,5,6] with the underlined elements coming from nums1.

```
void merge(int* nums1, int nums1Size, int m, int* nums2, int nums2Size, int n) {
    int i = m - 1;
    int j = n - 1;
    int k = m + n - 1;

    while (i >= 0 && j >= 0) {
        if (nums1[i] > nums2[j])
            nums1[k--] = nums1[i--];
        else
            nums1[k--] = nums2[j--];
    }

    while (j >= 0) {
        nums1[k--] = nums2[j--];
    }
}
```

OUTPUT:


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums1 =
[1,2,3,0,0,0]
m =
3
nums2 =
[2,5,6]
n =
3
Output
[1,2,2,3,5,6]


Problem List
Submit
Description
Editorial
Solutions
Submissions
Accepted
All Submissions
Accepted 59 / 59 testcases passed
MALYAVANTHAMSAIPRANEETHA submitted at Jun 02, 2026 17:29
Analysis
Solution
Runtime
0 ms | Beats 100.00%
Memory
10.78 MB | Beats 29.88%
150%
100%
50%
0%
1ms
2ms
3ms
4ms
Code | C
1 void merge(int* nums1, int nums1Size, int m, int* nums2, int nums2Size, int

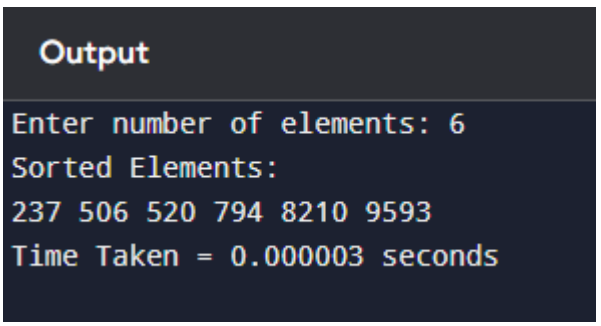
Lab Program 4:

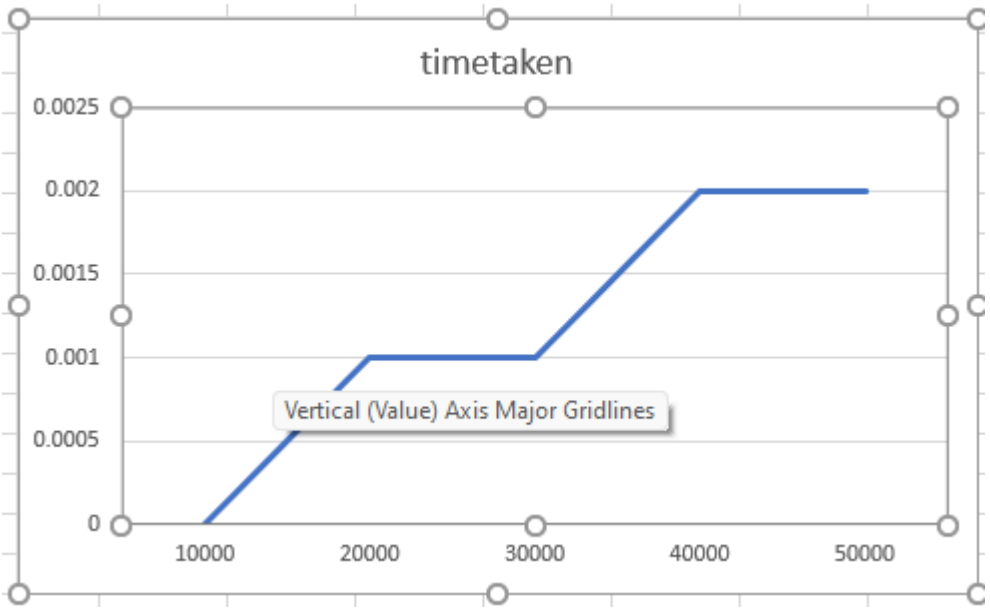
Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void swap(int *a, int *b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}

int partition(int arr[], int low, int high)
{
   int pivot = arr[high];
   int i = low - 1;
   for (int j = low; j < high; j++)
   {
      if (arr[j] <= pivot)
      {
         i++;
         swap(&arr[i], &arr[j]);
      }
   }
   swap(&arr[i + 1], &arr[high]);
   return i + 1;
}
void quickSort(int arr[], int low, int high)
{
   if (low < high)
   {
      int p = partition(arr, low, high);
      quickSort(arr, low, p - 1);
```

```
        quickSort(arr, p + 1, high);
    }
}
int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    srand(time(NULL));
    for (int i = 0; i < n; i++)
        arr[i] = rand() % 10000;
    clock_t start = clock();
    quickSort(arr, 0, n - 1);
    clock_t end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Sorted Elements:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\nTime Taken = %f seconds\n", time_taken);
    return 0;
}
```

OUTPUT:




timetaken
0.0025
0.002
0.0015
0.001
0.0005
0
10000
20000
30000
40000
50000
Vertical (Value) Axis Major Gridlines

455.Assign Cookies

455. Assign Cookies

Easy | Topics | Companies

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

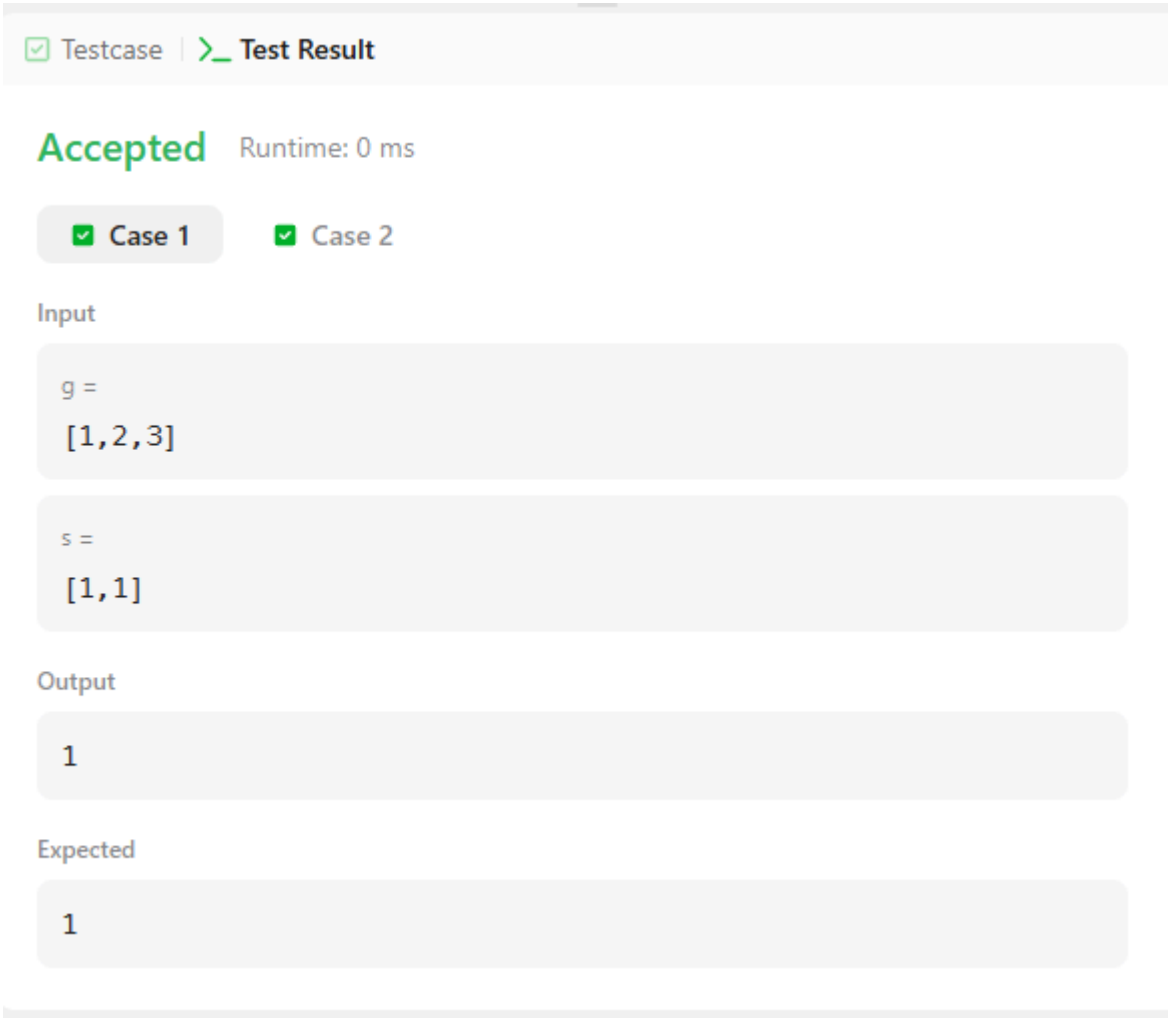
Each child `i` has a greed factor `g[i]`, which is the minimum size of a cookie that the child will be content with; and each cookie `j` has a size `s[j]`. If `s[j] >= g[i]`, we can assign the cookie `j` to the child `i`, and the child `i` will be content. Your goal is to maximize the number of your content children and output the maximum number.

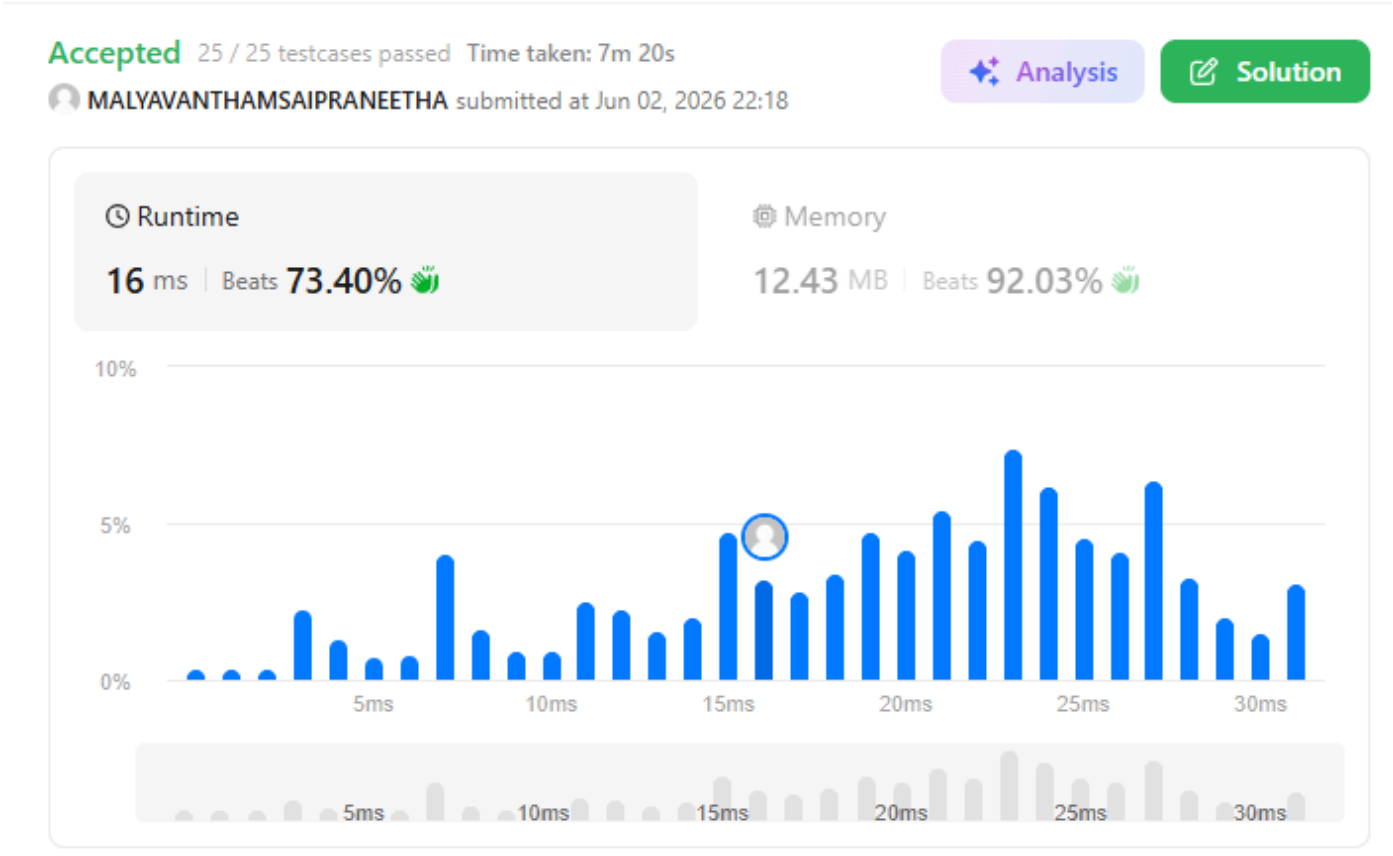
Example 1:

> **Input:** g = [1,2,3], s = [1,1]
> **Output:** 1
> **Explanation:** You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.
> And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.
> You need to output 1.

```c
#include <stdlib.h>
int compare(const void *a, const void *b)
{
    return (*(int *)a - *(int *)b);
}
int findContentChildren(int* g, int gSize, int* s, int sSize)
{
    qsort(g, gSize, sizeof(int), compare);
    qsort(s, sSize, sizeof(int), compare);
    int i = 0, j = 0;
    while (i < gSize && j < sSize)
    {
        if (s[j] >= g[i])
        {
            i++;
        }
        j++;
    }

    return i;
}
```

OUTPUT:


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
g =
[1,2,3]
s =
[1,1]
Output
1
Expected
1


Accepted
25 / 25 testcases passed
Time taken: 7m 20s
MALYAVANTHAMSAIPRANEETHA submitted at Jun 02, 2026 22:18
Analysis
Solution
Runtime
16 ms
Beats 73.40%
Memory
12.43 MB
Beats 92.03%
10%
5%
0%
5ms
10ms
15ms
20ms
25ms
30ms

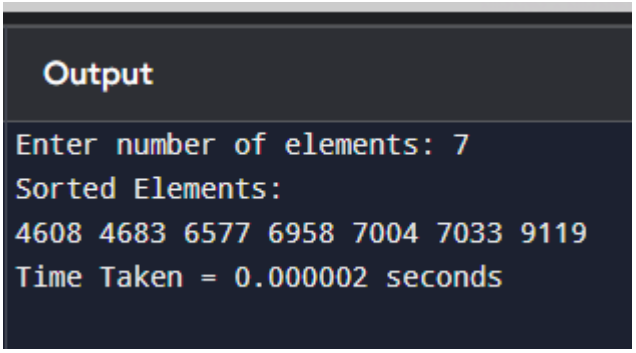
Lab Program 5:

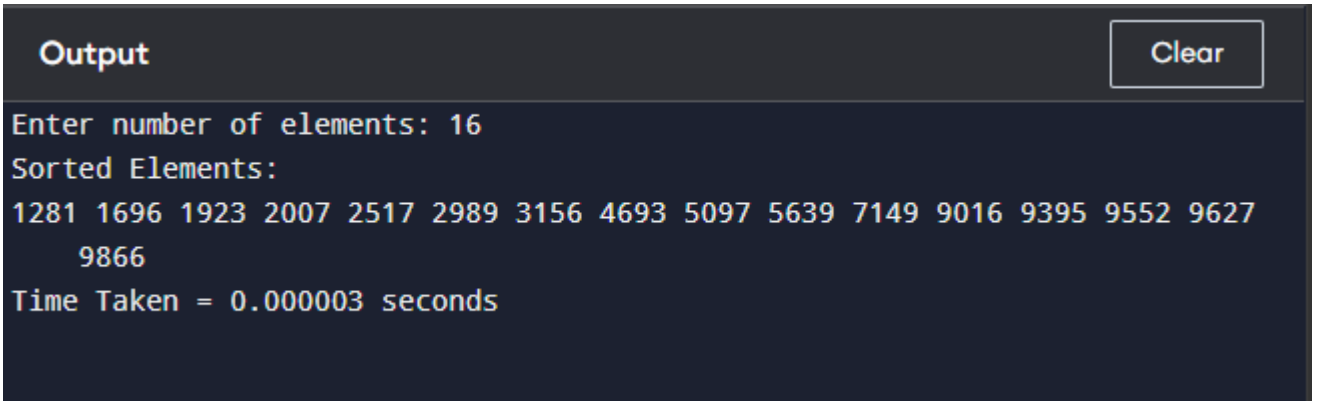
Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;
    for (int i = n - 1; i > 0; i--)
    {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```
    int arr[n];
    srand(time(NULL));
    for (int i = 0; i < n; i++)
        arr[i] = rand() % 10000;
    clock_t start = clock();
    heapSort(arr, n);

    clock_t end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Sorted Elements:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\nTime Taken = %f seconds\n", time_taken);

    return 0;
}
```

OUTPUT:





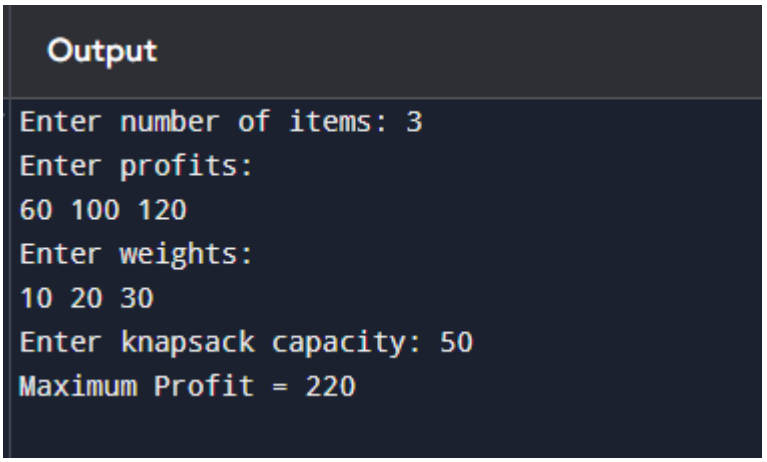
Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

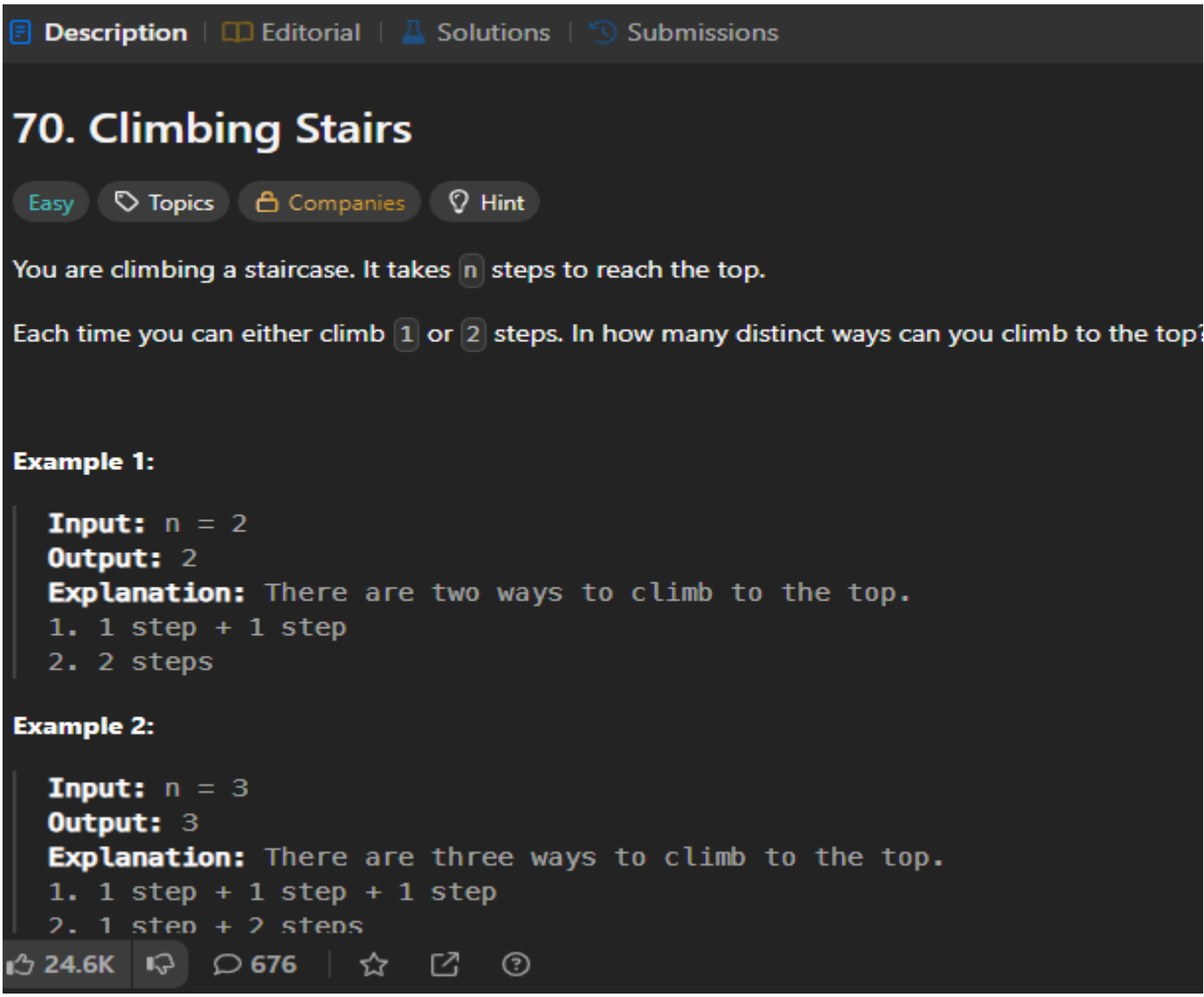
```
#include <stdio.h>
int max(int a, int b)
{
   return (a > b) ? a : b;
}
int knapsack(int W, int wt[], int val[], int n)
{
   int dp[n + 1][W + 1];
   for (int i = 0; i <= n; i++)
   {
      for (int w = 0; w <= W; w++)
      {
         if (i == 0 || w == 0)
            dp[i][w] = 0;
         else if (wt[i - 1] <= w)
            dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                           dp[i - 1][w]);
         else
            dp[i][w] = dp[i - 1][w];
      }
   }
   return dp[n][W];
}
int main()
{
   int n, W;
   printf("Enter number of items: ");
   scanf("%d", &n);
   int val[n], wt[n];
   printf("Enter profits:\n");
   for (int i = 0; i < n; i++)
```

```
        scanf("%d", &val[i]);
    printf("Enter weights:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);
    printf("Enter knapsack capacity: ");
    scanf("%d", &W);
    printf("Maximum Profit = %d\n", knapsack(W, wt, val, n));
    return 0;
}
```

OUTPUT:



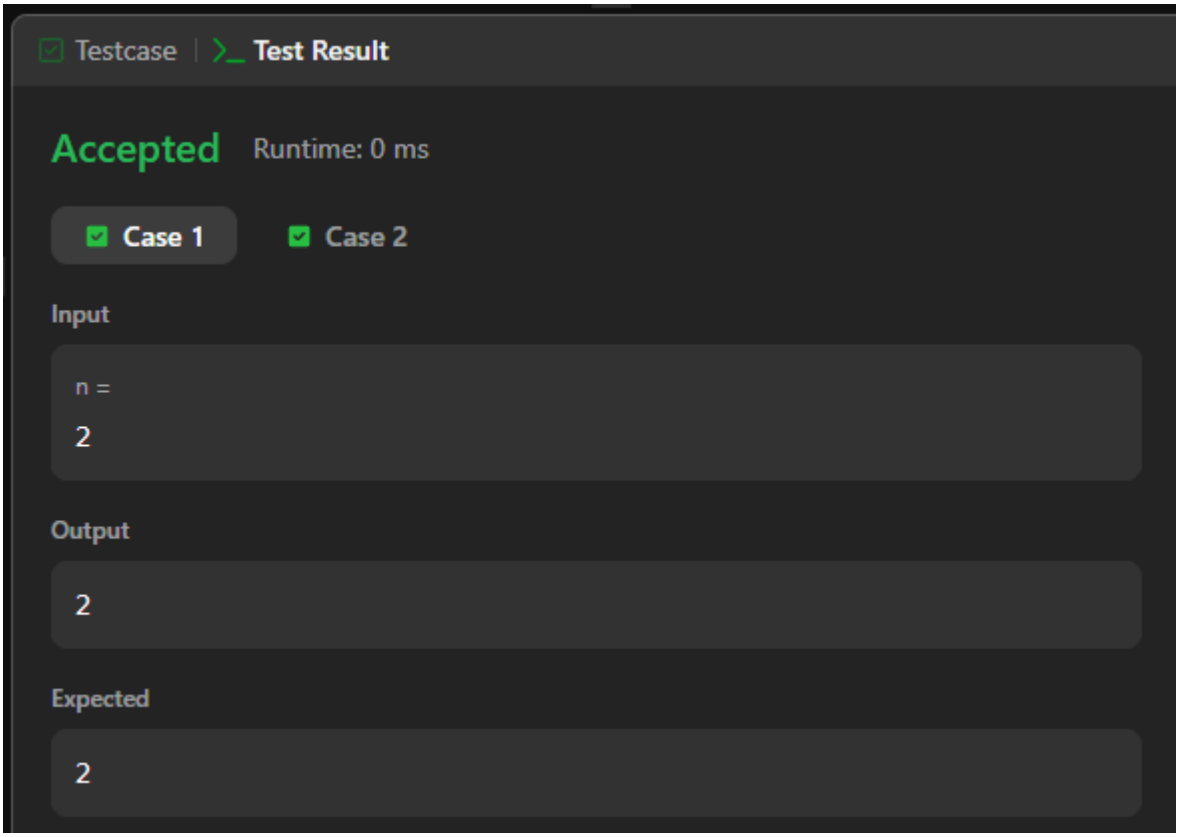
70.Climbing Stairs




```
int climbStairs(int n) {
    if(n<=2){
        return n;
    }
    int prev1=2;
    int prev2=1;
    int current;
    for(int i=3;i<=n;i++){
        current=prev1+prev2;
        prev2=prev1;
        prev1=current;

}
```

```
return prev1;
}
```

OUTPUT:



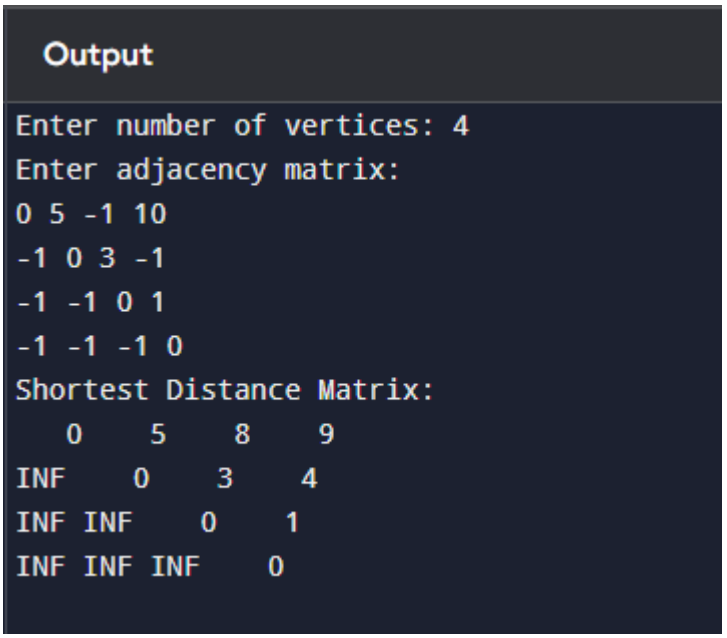
Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>
#define INF 99999
void floydWarshall(int graph[10][10], int n)
{
   int dist[10][10];
   for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
         dist[i][j] = graph[i][j];
   for (int k = 0; k < n; k++)
   {
      for (int i = 0; i < n; i++)
      {
         for (int j = 0; j < n; j++)
         {
            if (dist[i][k] + dist[k][j] < dist[i][j])
               dist[i][j] = dist[i][k] + dist[k][j];
         }
      }
   }
   printf("Shortest Distance Matrix:\n");
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         if (dist[i][j] == INF)
            printf("INF ");
         else
            printf("%4d ", dist[i][j]);
      }
      printf("\n");
   }
```

```
}
int main()
{
    int n;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int graph[10][10];
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);

            if (graph[i][j] == -1)
                graph[i][j] = INF;
        }
    }
    floydWarshall(graph, n);
    return 0;
}
```

OUTPUT:



LeetCode:733 Flood Fill

733. Flood Fill

Solved

Easy | Topics | Companies | Hint

You are given an image represented by an `m x n` grid of integers `image`, where `image[i][j]` represents the pixel value of the image. You are also given three integers `sr`, `sc`, and `color`. Your task is to perform a **flood fill** on the image starting from the pixel `image[sr][sc]`.

To perform a **flood fill**:

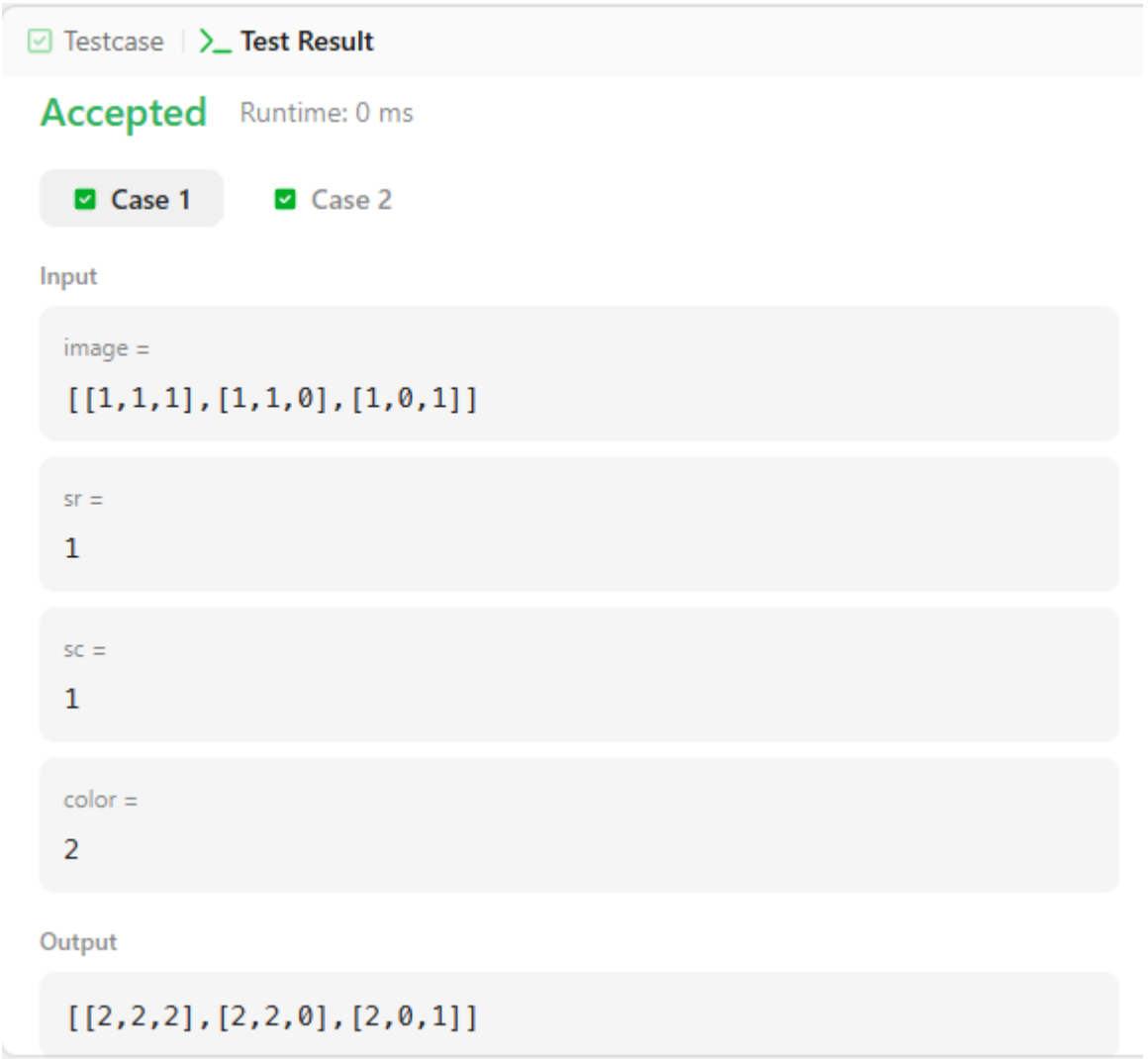
1. Begin with the starting pixel and change its color to `color`.
2. Perform the same process for each pixel that is **directly adjacent** (pixels that share a side with the original pixel, either horizontally or vertically) and shares the **same color** as the starting pixel.
3. Keep **repeating** this process by checking neighboring pixels of the *updated* pixels and modifying their color if it matches the original color of the starting pixel.
4. The process **stops** when there are **no more** adjacent pixels of the original color to update.

Return the **modified** image after performing the flood fill.

```
void dfs(int** image, int rows, int cols,
         int r, int c,
         int oldColor, int newColor)
{
    if(r < 0 || r >= rows || c < 0 || c >= cols)
        return;

    if(image[r][c] != oldColor)
        return;

    image[r][c] = newColor;

    dfs(image, rows, cols, r + 1, c, oldColor, newColor);
    dfs(image, rows, cols, r - 1, c, oldColor, newColor);
    dfs(image, rows, cols, r, c + 1, oldColor, newColor);
    dfs(image, rows, cols, r, c - 1, oldColor, newColor);
}

int** floodFill(int** image, int imageSize, int* imageColSize,
                int sr, int sc, int color,
                int* returnSize, int** returnColumnSizes)
```

```
{
    int oldColor = image[sr][sc];

    *returnSize = imageSize;
    *returnColumnSizes = imageColSize;

    if(oldColor == color)
        return image;

    dfs(image, imageSize, imageColSize[0],
        sr, sc, oldColor, color);

    return image;
}
```

OUTPUT:



Accepted 278 / 278 testcases passed

MALYAVANTHAMSAIPRANEETHA submitted at Jun 01, 2026 09:59

Runtime

0 ms | Beats **100.00%**

Memory

13.65 MB | Beats **88.69%**

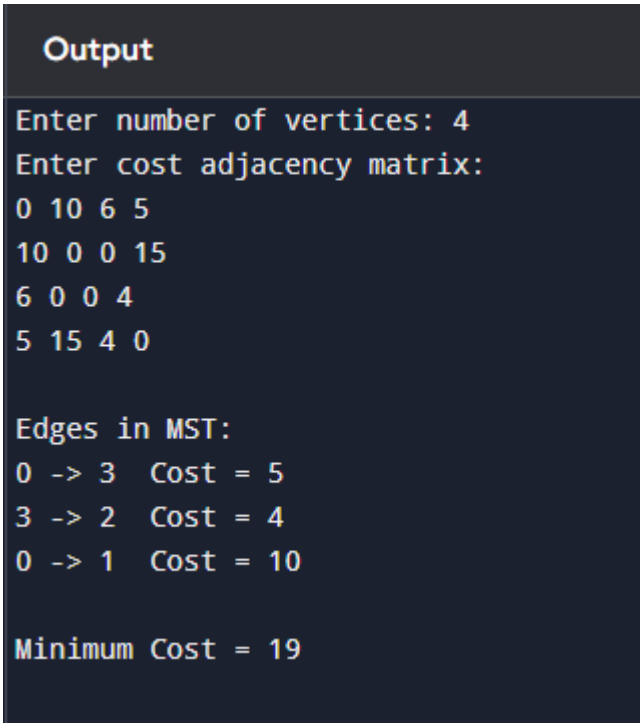
Lab Program 8a:

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>
#define INF 9999
int main()
{
   int n;
   printf("Enter number of vertices: ");
   scanf("%d", &n);
   int cost[10][10];
   printf("Enter cost adjacency matrix:\n");
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0)
            cost[i][j] = INF;
      }
   }
   int visited[10] = {0};
   visited[0] = 1;
   int edges = 0, mincost = 0;
   printf("\nEdges in MST:\n");
   while (edges < n - 1)
   {
      int min = INF;
      int a = -1, b = -1;
      for (int i = 0; i < n; i++)
      {
         if (visited[i])
         {
```

```
            for (int j = 0; j < n; j++)
            {
               if (!visited[j] && cost[i][j] < min)
               {
                  min = cost[i][j];
                  a = i;
                  b = j;
               }
            }
         }
      }
      printf("%d -> %d  Cost = %d\n", a, b, min);
      mincost += min;
     visited[b] = 1;
      edges++;
   }
   printf("\nMinimum Cost = %d\n", mincost);
   return 0;
}
```

OUTPUT:


Output
Enter number of vertices: 4
Enter cost adjacency matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0
Edges in MST:
0 -> 3 Cost = 5
3 -> 2 Cost = 4
0 -> 1 Cost = 10
Minimum Cost = 19

8b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>
#define MAX 20
struct Edge
{
   int u, v, w;
};
int parent[MAX];
int find(int i)
{
   while (parent[i] != i)
      i = parent[i];
   return i;
}
void unionSet(int a, int b)
{
   parent[a] = b;
}
int main()
{
   int n, e;
   printf("Enter number of vertices and edges: ");
   scanf("%d %d", &n, &e);
   struct Edge edge[MAX];
   printf("Enter edges (u v weight):\n");
   for (int i = 0; i < e; i++)
      scanf("%d %d %d", &edge[i].u, &edge[i].v, &edge[i].w);
   for (int i = 0; i < n; i++)
      parent[i] = i;
   for (int i = 0; i < e - 1; i++)
   {
      for (int j = 0; j < e - i - 1; j++)
```

```
        {
            if (edge[j].w > edge[j + 1].w)
            {
                struct Edge temp = edge[j];
                edge[j] = edge[j + 1];
                edge[j + 1] = temp;
            }
        }
    }
    int mincost = 0;
    printf("\nEdges in MST:\n");
    for (int i = 0; i < e; i++)
    {
        int u = find(edge[i].u);
        int v = find(edge[i].v);
        if (u != v)
        {
            printf("%d -- %d = %d\n",
                   edge[i].u, edge[i].v, edge[i].w);
            mincost += edge[i].w;
            unionSet(u, v);
        }
    }
    printf("\nMinimum Cost = %d\n", mincost);
    return 0;
}
```

OUTPUT:

```
Output

Enter number of vertices and edges: 4 5
Enter edges (u v weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Edges in MST:
2 -- 3 = 4
0 -- 3 = 5
0 -- 1 = 10

Minimum Cost = 19
```

```
Output

Enter number of vertices and edges: 3 3
Enter edges (u v weight):
0 1 1
1 2 2
0 2 3

Edges in MST:
0 -- 1 = 1
1 -- 2 = 2

Minimum Cost = 3
```

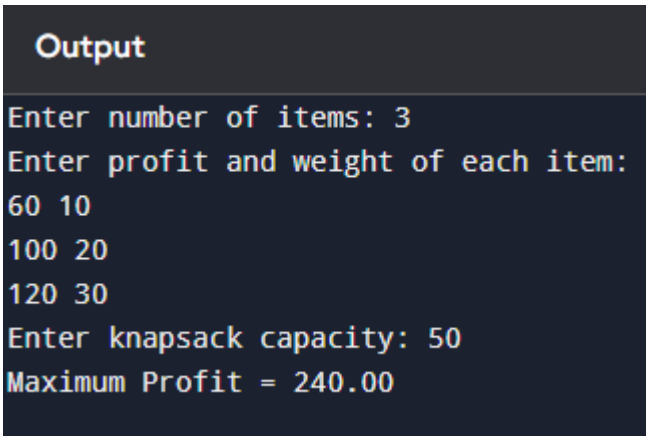
Lab Program 9:

Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>
struct Item
{
    int profit;
    int weight;
    float ratio;
};
int main()
{
    int n, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
    struct Item item[n];
    printf("Enter profit and weight of each item:\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d %d", &item[i].profit, &item[i].weight);
        item[i].ratio = (float)item[i].profit / item[i].weight;
    }
    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (item[j].ratio < item[j + 1].ratio)
            {
                struct Item temp = item[j];
                item[j] = item[j + 1];
                item[j + 1] = temp;
            }
```

```
        }
    }

    float totalProfit = 0.0;
    for (int i = 0; i < n; i++)
    {
        if (capacity >= item[i].weight)
        {
            totalProfit += item[i].profit;
            capacity -= item[i].weight;
        }
        else
        {
            totalProfit += item[i].ratio * capacity;
            break;
        }
    }
    printf("Maximum Profit = %.2f\n", totalProfit);
    return 0;
}
```

OUTPUT:



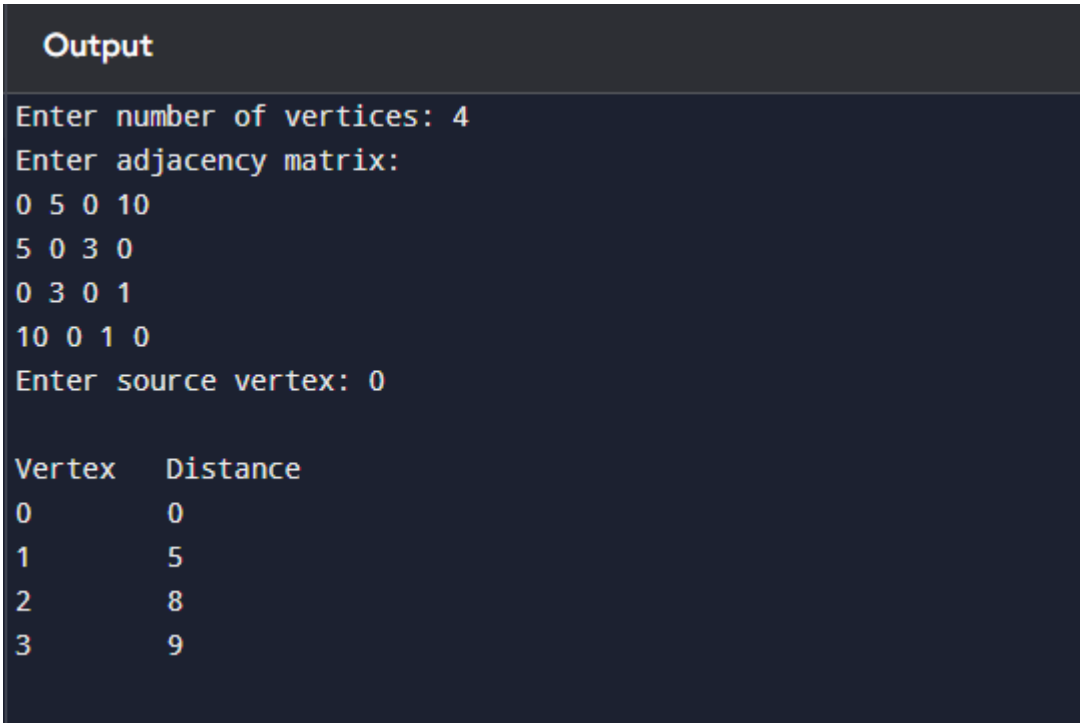
Lab Program - 10

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>
#define INF 9999
int main()
{
   int n;
   printf("Enter number of vertices: ");
   scanf("%d", &n);
   int cost[10][10];
   printf("Enter cost adjacency matrix:\n");
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         scanf("%d", &cost[i][j]);

         if (cost[i][j] == 0 && i != j)
            cost[i][j] = INF;
      }
   }
   int source;
   printf("Enter source vertex: ");
   scanf("%d", &source);
   int dist[10], visited[10];
   for (int i = 0; i < n; i++)
   {
      dist[i] = cost[source][i];
      visited[i] = 0;
   }
   dist[source] = 0;
   visited[source] = 1;
```

```
    for (int count = 1; count < n; count++)
    {
        int min = INF, u = -1;
        for (int i = 0; i < n; i++)
        {
            if (!visited[i] && dist[i] < min)
            {
                min = dist[i];
                u = i;
            }
        }
        visited[u] = 1;
        for (int v = 0; v < n; v++)
        {
            if (!visited[v] &&
                dist[u] + cost[u][v] < dist[v])
            {
                dist[v] = dist[u] + cost[u][v];
            }
        }
    }
    printf("\nShortest distances from vertex %d:\n", source);
    for (int i = 0; i < n; i++)
        printf("%d -> %d = %d\n", source, i, dist[i]);
    return 0;
}
```

OUTPUT:


Output
Enter number of vertices: 4
Enter adjacency matrix:
0 5 0 10
5 0 3 0
0 3 0 1
10 0 1 0
Enter source vertex: 0
Vertex Distance
0 0
1 5
2 8
3 9

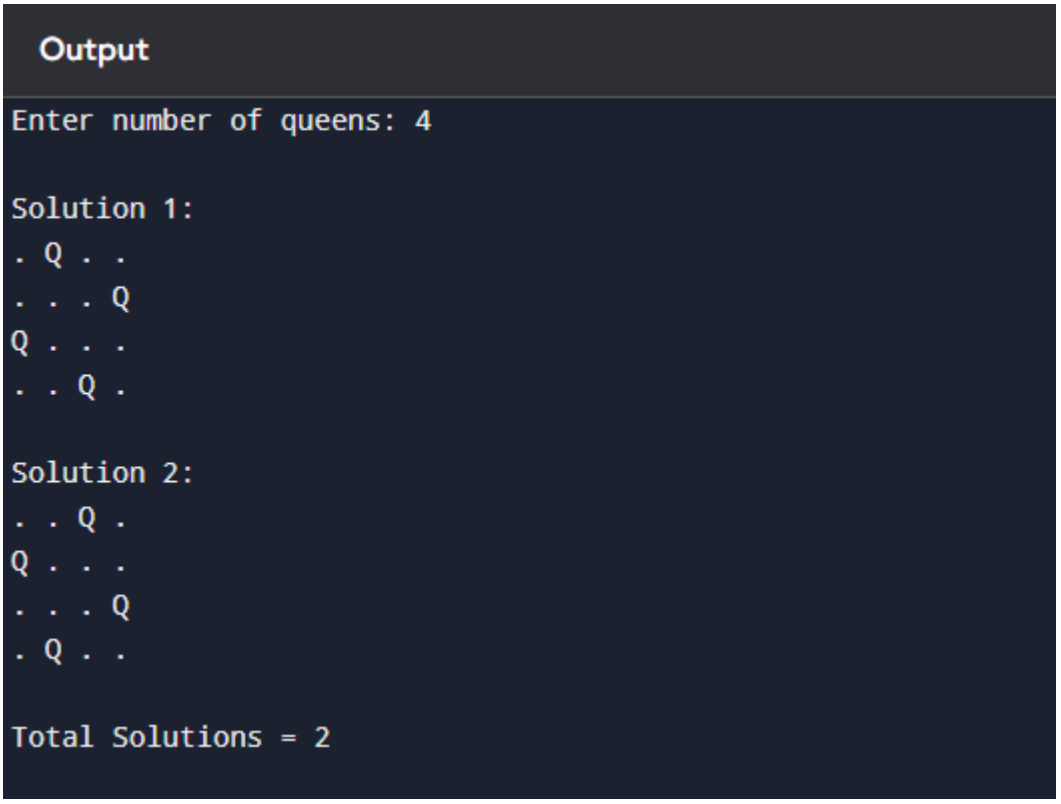
Lab Program 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
int x[20], count = 0;
int place(int k, int i)
{
   for (int j = 1; j < k; j++)
   {
      if (x[j] == i || abs(x[j] - i) == abs(j - k))
         return 0;
   }
   return 1;
}
void nQueens(int k, int n)
{
   for (int i = 1; i <= n; i++)
   {
      if (place(k, i))
      {
         x[k] = i;
         if (k == n)
         {
            count++;
            printf("\nSolution %d:\n", count);
            for (int r = 1; r <= n; r++)
            {
               for (int c = 1; c <= n; c++)
               {
                  if (x[r] == c)
                     printf("Q ");
                  else
                     printf(". ");
               }
```

```
                printf("\n");
            }
        }
        else
        {
            nQueens(k + 1, n);
        }
    }
  }
}
int main()
{
  int n;
  printf("Enter number of queens: ");
  scanf("%d", &n);
  nQueens(1, n);
  printf("\nTotal Solutions = %d\n", count);
  return 0;
}
```

OUTPUT:


Output
Enter number of queens: 4
Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .
Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .
Total Solutions = 2